# On-Device Learning and ESP32 Streaming Summary

## 1. Core idea

- In this project, the ESP32 mainly performs on-device inference using a pretrained TinyML model built from 76 engineered vibration features.
- On-device learning does not mean the ESP32 should blindly train on every new incoming sample. Inference is automatic, but training requires a separate update policy.
- The device does not automatically know the true label of a new sample. Ground truth must come from a human operator, another trusted system, or an unlabeled adaptation strategy.
- A safer design is: predict every sample, store only selected samples, and update the model only periodically or under controlled conditions.

## 2. What happens when new data comes to the ESP32

- A new vibration sample or feature vector arrives at the ESP32.
- The ESP32 first uses the current model for prediction.
- Then a decision policy determines whether the sample is only used for inference, stored for later review, or used in a controlled model update.
- Most samples should be used only for inference. Only selected samples should be buffered for adaptation or retraining.

## 3. How the device knows whether to predict or train

- The device does not naturally know whether a sample should be used for prediction or training. This must be defined by a learning policy.
- Typical policies are: inference only unless a trusted label arrives; store uncertain samples; retrain only after collecting a batch; or update only anomaly thresholds instead of the full classifier.
- Training on every sample is usually avoided because labels are often unknown, memory is limited, and wrong predictions can reinforce errors.

## 4. Where the correct label comes from

- For supervised learning, the ESP32 needs an external source of truth.
- That label may come from a human operator, a maintenance engineer, a supervisory system, or a controlled test setup.
- Without such a source, the device should not blindly retrain on its own predictions because it may learn from mistakes and drift away from correct behavior.

## 5. Can the device learn without labels

- Yes, but then it is not standard supervised classification training.
- The device can adapt healthy-reference statistics, anomaly thresholds, running means, or simple prototype models without explicit labels.

- For this pump project, lightweight adaptation of healthy baseline behavior is more realistic than full continuous retraining of the TinyML classifier on the ESP32.

## 6. Recommended approach for this project

- Use the ESP32 mainly for inference, confidence estimation, buffering of important samples, and lightweight adaptation.
- Use periodic supervised retraining on a laptop or backend once trusted labels are available.
- Redeploy the improved model to the ESP32 instead of letting it continuously retrain itself on every sample.

## 7. Successful streaming result implemented in this project

- A working streaming pipeline was successfully implemented between laptop and ESP32.
- One data point is sent from the laptop to the ESP32, the ESP32 acknowledges receipt, performs inference using the already deployed model, returns the prediction, deletes the streamed point, stores the result in a stream log, and acknowledges readiness for the next point.
- This process was repeated for 1000 streamed points and achieved 96.2% accuracy on the ESP32.
- The log file created during streaming is stream_resultslog.csv. Since it is only needed for collecting results, it can be deleted afterwards and should not be counted as necessary persistent model memory.

## 8. Memory results from streaming experiment

- Initial memory footprint on ESP32 after loading the model: 28.45 KB.
- Memory footprint after streaming 1000 points and storing logs: about 65 KB.
- Because the log file can be removed after extracting results, the larger value reflects temporary experiment logging overhead rather than permanent model size alone.

## 9. Final conclusion

- In this project, on-device learning should be understood as controlled adaptation, not blind retraining on every incoming sample.
- The ESP32 can always perform inference on new data, but deciding whether that data should also be used for learning requires a rule, a label source, or a safe unlabeled adaptation strategy.
- The current streaming experiment already demonstrates a strong practical result: reliable on-device inference with 96.2% accuracy on 1000 streamed points using a lightweight ESP32 deployment workflow.